

BST Number List in Console/Terminal
Cpt S 321 Homework Assignment
Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Before you do anything, create a folder in your main branch (of your HW repository) called "AI_Statements"
2. Create a branch called "Branch_HW1" and work in this branch for this assignment.
3. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
4. **Work on the Assignment Instructions below.**
5. **Create a pdf document with your AI statement for the HW (call it "AI_HW1"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.**
6. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW1", "HW2", etc.
7. **Record a video to demo your HW (call it "Demo_HW1-FirstName_LastName").**
8. On Canvas -> Assignments -> Assignments -> **Upload your pre-recorded video by the HW deadline.**
9. Sign up for a live demo with the TAs before the deadline.

Assignment Instructions:

Read all the instructions *carefully* before you write any code.

Create a C# console application that fulfills the following requirements:

1. Get a list of integer numbers from the user on A SINGLE LINE
 - The numbers will be in the range [0,100]
 - The numbers will be separated by spaces
 - You may assume that the user enters a correctly formatted input string that meets these requirements
 - You may use [Console.ReadLine](#) or a similar method to get input from the user
2. Add all the numbers to a binary search tree (you must implement your own BST) in the order they were entered
 - Don't allow duplicates

- Use the [Split](#) function on the entered string for easy parsing (split on the space character)
3. Display the numbers in sorted order (smallest first, largest last).
 - Traverse the tree in order to produce this output.
 4. Display the following statistics about the tree
 - Number of items (note that this will be less than or equal to the number of items entered by the user, since duplicates won't be added to the tree). Write a function that determines this from your BST, NOT the array returned from the split. In other words, you must have a Count function in your BST implementation (you are not allowed to use any existing implementation for that).
 - Number of levels in the tree. A tree with no nodes at all has 0 levels. A tree with a single node has 1 level. A tree with 2 nodes has 2 levels. A tree with three nodes could have 2 or 3 levels. You should know why this is from your advanced data structures prerequisite course.
 - Theoretical minimum number of levels that the tree could have given the number of nodes it contains (figure out the formula to calculate this)

Sample Output:

```

C:\Windows\system32\cmd.exe
Enter a collection of numbers in the range [0, 100], separated by spaces:
55 22 77 88 11 22 44 77 55 99 22
Tree contents: 11 22 44 55 77 88 99
Tree statistics:
  Number of nodes: 7
  Number of levels: 4
  Minimum number of levels that a tree with 7 nodes could have = 3
Done
  
```

The ~~strikethrough~~ text below means that this is something that we will be adding in later assignments but you are free to skim through and work on them if you feel up for the task.

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	• Fulfill all the requirements above with no inaccuracies in the output and no crashes.
CR2 Quality of Version Control	• Homework should be built iteratively (i.e., one feature at

(1 point)	a time, not in one huge commit). • Each commit should have cohesive functionality. • Commit messages should concisely describe what is being committed. • TDD should be used (i.e, write and commit tests first and then implement and commit functionality). • Include "TDD" in all commit messages with tests that are written before the functionality is implemented. • Use of a .gitignore. • Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	• No underscores in names of classes, attributes, and properties. • No numbers in names of classes or tests. • Identifiers should be descriptive. • Project names should make sense. • Class names and method names use PascalCasing. • Method arguments and local variables use camelCasing. • No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	• Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. • All comment blocks use the format that is generated when typing "///" on the line above each entity. • There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	• Each file should only contain one public class. • Correct use of access modifiers. • Classes are cohesive. • Namespaces make sense. • Code is easy to follow. • StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. • Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	• Normal, boundary, and overflow/error cases should be tested for each feature and should be documented as comments in the code. • Test cases should be written with NUnit framework.

	● Test should be modularized (i.e, have a separate test case for each functionality – do not combine them into one large test case). ● Note: In assignments with a GUI, we do not require testing of the GUI itself.
CR7: Demo video – 5 min max (1 point)	● Check the “Pre-recorded demos instructions” in the additional resources for details on the items below. ● Version control history overview (< 30 sec.) ● AI statement (< 30 sec.) ● Design of the solution (2 min max) ● Features showcase (2 min max)